Economic-Aware and Sovereignty-Constrained Routing for Enterprise LLM Gateways

(authors omitted for review)

Draft v0.4 — two-provider (OpenAI US + Mistral EU) evaluation with a 30-repetition statistical run

Abstract

Enterprises are adopting large language models (LLMs) faster than they can govern them: they cannot say which model truly costs the most, which team consumes what, when to route to a cheaper model, what a data-sovereignty constraint costs, or when self-hosting beats a managed API. We present an *economic-aware, sovereignty-constrained, budget-aware routing control plane* for enterprise LLM gateways. We formalize model selection as a constrained scoring problem that rejects any model violating declarative sovereignty rules and, among the rest, minimizes a weighted combination of cost, expected quality loss, latency, and budget pressure. We realize the approach as a Kubernetes operator whose FinOps and sovereignty engines drive routing, and evaluate it on a reproducible testbed with **real LLM calls across two providers** — OpenAI (US, managed) and **Mistral (EU)** served via Azure AI Foundry (Mistral-Large-3, *DataZoneStandard* EU data zone). Relative to an always-premium policy, our approach reduces measured cost by **70.8%** while keeping LLM-as-judge quality comparable to premium; a **30-repetition** run (temperature 0.7) confirms the cost reduction is highly significant ($p \approx 3 \times 10^{-11}$, Cohen $d \approx -43$) at a significant but bounded latency increase. It enforces five declarative sovereignty scenarios with **zero violations** (vs. 40 for a sovereignty-blind baseline) and keeps **100% availability under an EU-only policy** via the real EU provider, and sustains 100% request availability with 0% budget overrun under a tight budget where hard-blocking drops to 60%. On quality we are deliberately careful: on **500 objective public-benchmark items** (GSM8K + MMLU) our policy *trades accuracy for cost* (47.2% vs. 65.4% premium, −94% cost), and we show the LLM judge *over-rates incorrect answers* (81.1% of wrong answers rated "acceptable"). We frame the contribution as a **governance layer** with a favourable cost/quality trade-off, not a quality-free lunch, and release all code, datasets, cached responses, and analysis as a reproducible framework.

1 Introduction

LLMs are now embedded in HR assistants, document RAG, developer tooling, and analytical agents. Each call has a price that depends on the model, the provider, and the token volume; each call may also send data across jurisdictions. Three problems recur in regulated enterprises:

1. **Cost opacity.** Teams cannot attribute spend per application/team or decide when a cheaper model would suffice.
2. **Sovereignty/compliance.** Data may not leave a permitted zone, and sensitive prompts may not go to external providers; today this is enforced ad hoc, if at all.

3. **Managed vs. self-hosted arbitrage.** It is unclear at what volume self-hosting on GPU becomes cheaper than a managed API.

Existing gateways and FinOps tools mostly *measure and report*. Our thesis is that a control plane at the **gateway** can *act*: route each request to the cheapest model that meets a minimum quality bar and all hard sovereignty constraints, degrade gracefully under budget pressure instead of blocking, and surface the self-hosting break-even from observed usage.

Contributions.

- A constrained scoring formulation for LLM routing unifying cost, quality, latency, sovereignty, and budget (§3).
- A Kubernetes/Envoy control-plane realization — the *AI Sovereign FinOps Operator* with Cost, Budget, Sovereignty, Break-even, and Report engines exposed as CRDs (§4).
- A reproducible evaluation methodology and testbed driving the operator's actual engines with real LLM telemetry (§5).
- Empirical evidence across six research questions (§6), plus an objective public-benchmark study and an LLM-judge validity analysis.

2 Background and Related Work

Model routing and cascades. A growing line of work trades quality for cost by selecting among models per request. FrugalGPT [3] cascades from cheap to expensive models using a learned scorer; Hybrid LLM [4] routes by predicted query difficulty with a tunable quality target; RouteLLM [10] learns routers from preference data; AutoMix [1] uses few-shot self-verification and a POMDP router. These works optimize a *cost–quality* trade-off, largely via per-request difficulty/confidence prediction for a single tenant. Our work is **orthogonal and complementary**: they decide *which model is good enough*; we add the **governance layer** around that decision — hard data-sovereignty constraints, per-team/namespace budgets, budget-aware graceful degradation, attribution, and hosting economics — at the gateway/control-plane level. Our routing score is intentionally simple (priors, not a learned difficulty predictor); combining a learned difficulty router with our sovereignty/budget constraints is explicit future work (§9).

LLM serving systems. Orca [11], vLLM/PagedAttention [8], SGLang [13], and DistServe [14] optimize *throughput/latency* of a single self-hosted engine; surveys [9, 15] organize this space. They are complementary: they define the *cost* of the self-hosted option whose break-even our system predicts (RQ6), but do not address cross-provider routing, attribution, or sovereignty.

LLM gateways. Envoy AI Gateway [5] and LiteLLM [2] provide a unified data path to multiple providers and expose usage telemetry, but leave *policy* (which model, under which constraints, within which budget) to the operator. We contribute that policy layer as a Kubernetes control plane and validate it.

Evaluation and regulation. We follow the LLM-as-a-judge methodology of Zheng et al. [12], which shows strong judges approximate human preference at ∼80% agreement, adopting both absolute and pairwise protocols while acknowledging its documented biases (§8). Data-residency

and sensitive-data handling are driven by the GDPR [6] and the EU AI Act [7]; our system targets *audit preparation*, not legal attestation.

Table 1: Positioning feature matrix. Prior routing work optimizes the cost–quality trade-off; we add the governance dimensions at the control-plane level.

Capability	FrugalGPT	Hybrid LLM	RouteLLM	AutoMix	**Ours**
Cost-aware routing	✓	✓	✓	✓	✓
Quality-aware routing	✓	✓	✓	✓	✓
Learned difficulty predictor	✓	✓	✓	✓	✗ (priors)
Hard sovereignty	✗	✗	✗	✗	✓
Per-tenant budgets	✗	✗	✗	✗	✓
Graceful degradation	✗	✗	✗	✗	✓
Cost attribution	✗	✗	✗	✗	✓
Break-even (mgd/self)	✗	✗	✗	✗	✓
Gateway/K8s artifact	✗	✗	✗	✗	✓

We do not claim to beat learned routers on the cost–quality frontier; our governance layer is *orthogonal* and could wrap any of them.

Table 2: Side-by-side vs. representative systems — three learned routers and a production gateway.

Dimension	RouteLLM	FrugalGPT	AutoMix	LiteLLM	**Ours**
Type	router	cascade	router	gateway	**ctrl plane**
Multi-provider	part.	✓	✓	✓	✓
Hard sovereignty	✗	✗	✗	✗	✓
Budget + degrad.	✗	✗	✗	basic	✓
Cost attribution	✗	✗	✗	per-key	✓
Break-even	✗	✗	✗	✗	✓
K8s/Envoy artifact	✗	✗	✗	proxy	✓

LiteLLM is the closest *engineering* peer (a real gateway with per-key spend and fallbacks) but offers no constraint-aware routing, graceful degradation, or break-even; the learned routers are stronger on the cost–quality decision but carry none of the governance dimensions. Our contribution is the governance layer, designed to compose with a learned router rather than replace it.

3 Problem Formulation and Algorithm

A request r carries metadata (team, namespace, sensitivity) and an estimated token footprint. Given a catalog of models M (each with provider, zone, managed flag, per-token price, quality prior, latency prior), a sovereignty policy P, and budget state $(used, total)$:

Hard constraint. Reject any m that violates P: its zone is forbidden; or an allow-list is set and m's zone is outside it (EU covers EU member states); or r is sensitive, m is a managed external provider, and external providers are disallowed for sensitive data. Let C be the surviving candidates. If $C = \varnothing$, the request is not served (a measured outcome, not a crash).

Soft objective. Among C meeting a minimum quality $q_{\min}$ (relaxed only when the budget is exhausted, enabling graceful degradation), select

$$\arg\min_{m \in C} \; \alpha\, \widehat{c}(m)\,(1 + \varepsilon\rho) + \beta\, \ell_q(m) + \gamma\, \ell_\lambda(m) + \delta\, \sigma(m), \tag{1}$$

where $\widehat{c}(m) = \text{cost}(m)/\text{cost}(\text{premium})$ is normalized cost, $\rho = \textit{used}/\textit{total}$ is budget pressure, $\ell_q(m) = \max(0, (q_{\text{prem}} - q_m)/q_{\text{prem}})$ is expected quality loss, ℓ_λ is normalized latency, and σ a residual sovereignty risk. Budget pressure multiplies the cost term, so as the budget fills the router shifts to cheaper models. Defaults: α=1.0, β=1.5, γ=0.3, δ=0.5, ε=1.0. The implementation is `experimentation/internal/router` (strategy `B6-ours`), reusing the operator's sovereignty normalization.

4 System Design

Requests flow client → **Envoy AI Gateway** → provider (managed API or self-hosted). The **AI Sovereign FinOps Operator** is the control plane: CRDs declare gateways, providers, models, budgets, and sovereignty policies; pure engines compute cost (per model/team/namespace), budget phase, sovereignty findings, and break-even; a report engine emits Markdown/JSON and Prometheus `ai_finops_*` metrics. The design keeps the data path in Envoy and the decision logic in the operator. In this paper the routing decision is evaluated at the control-plane level; data-path enforcement inside Envoy is the live-integration step (§9).

5 Methodology

Testbed. A Go harness drives the operator's *actual* engines and issues **real** LLM calls across **two providers**: (1) **OpenAI (US, managed)** — gpt-4o (premium), gpt-4o-mini (medium), gpt-4.1-nano (cheap); (2) **Mistral (EU)** — `mistral-large` served via **Azure AI Foundry** as **Mistral-Large-3** in *DataZoneStandard* (EU data zone), a real EU-hosted provider. A *modeled* EU self-hosted model (cost computed, response stubbed, rows tagged *modeled*) is retained only for the RQ6 break-even prediction. The harness is provider-agnostic (per-provider OpenAI-compatible clients), so adding providers is additive.

Workloads (W1–W4). HR chatbot (short, sensitive), RAG docs (long context, quality-sensitive), Dev assistant (high quality), Analytical agent (high volume). Each declares team, namespace, sensitivity, monthly budget, premium model, allowed models, and $q_{\min}$.

Baselines. B1 premium-static, B2 round-robin, B3 least-cost, B4 static namespace policy, B5 budget hard-block, B6 ours; plus B7 a literature-style difficulty router (Hybrid-LLM/RouteLLM proxy).

Metrics. Cost (total, per request, per token, per team via the operator cost engine, % savings); quality (LLM-as-judge acceptability 1–5 → normalized 0–1, acceptable-rate ≥ 3, pairwise win-rate vs. premium); latency (p50/p95/p99, mean, routing-decision μs); sovereignty (violations, reroutes, blocked); budget (availability, overrun); break-even (managed vs. self-hosted monthly, payback).

Reproducibility. Temperature 0 for the deterministic pass; all responses and judge scores cached so re-runs are identical and free; every test journaled with status+duration (42 PASS /

0 FAIL). Latency 95% CIs via bootstrap. The protocol also supports the $\geq$ 30-repetition run reported in §6.9.

6 Results

6.1 RQ1 — Cost

Relative to premium-static, **B6-ours cuts total cost by 70.8%** (0.01135 vs. 0.03893 EUR over the 40-prompt matrix; cost/request 0.000284 vs. 0.000973). With the second provider available, Ours routes much traffic to the cheaper Mistral EU and OpenAI mid/cheap tiers. A *literature-style difficulty router* (B7) saves slightly *more* (73.1%) but at lower quality (0.866) and with no sovereignty/budget/attribution governance. Naïve least-cost (B3) saves 98.2% at the largest quality cost; round-robin 62.3%; static policy 46.0% (Figure 1).

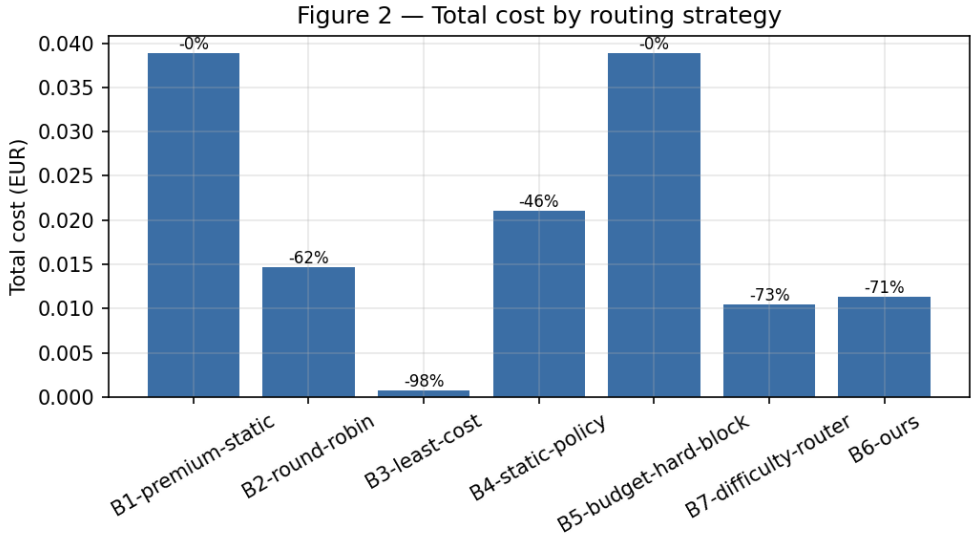

Figure 1: Total cost by routing strategy (lower is better).

6.2 RQ2 — Quality (LLM-as-judge)

Within this two-provider, single-pass scope, B6-ours's mean normalized quality is **comparable to — marginally above — premium** (0.9125 vs. 0.900; acceptable-rate 97.5%), with a pairwise win-rate vs. premium of **50.0%** (statistical parity). The cross-provider mix helps: Mistral-Large-3 sometimes matches or beats gpt-4o on the judge. Least-cost drops to 0.858, static-policy to 0.869, and the difficulty router (B7) to 0.866. Two caveats: (i) the premium model is also the routing reference and judge anchor, which can bias pairwise results; (ii) this is a single deterministic pass. We therefore claim only that *quality remained comparable within the evaluated scope* (Figure 2).

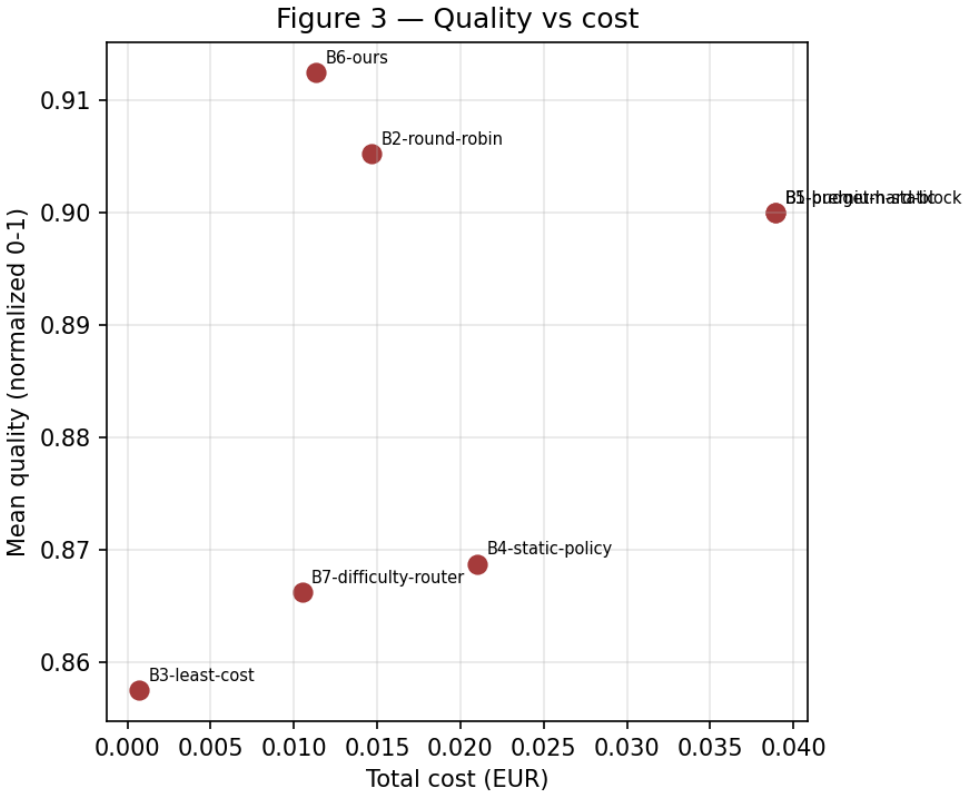

Figure 2: Quality vs. cost: Ours sits near premium quality at a fraction of the cost.

6.3 RQ2b — Objective accuracy + judge validity

To remove judge bias and test generality, we evaluate on **500 real public-benchmark items** — 250 **GSM8K** (math, numeric) and 250 **MMLU** (multiple-choice A–D) — scored by **objective exact-match**. Premium-static (gpt-4o) reaches **65.4%** accuracy at 0.116 EUR; **Ours 47.2%** at 0.0073 EUR (−94% **cost**); static-policy 59.4%; least-cost/difficulty-router 44–46%. On these objective tasks aggressive cost routing *trades accuracy for cost.*

Why this contradicts the judge (a methodological finding). We re-scored 300 objective answers with *both* exact-match and the LLM judge: the judge rated **81.1% of objectively *incorrect* answers as "acceptable" ($\geq$ 3)**, with nearly identical mean judge scores for correct (4.49) vs. incorrect (4.14) answers and near-zero point-biserial correlation (r=0.12). **The LLM-as-judge over-estimates quality on objective tasks** — it cannot distinguish fluent-but-wrong from correct (Figure 3). This explains the RQ2-vs-RQ2b gap, and we adopt the consequence throughout: report judge-based quality only for open-ended workloads, exact-match for tasks with ground truth, and never claim quality preservation from the judge alone.

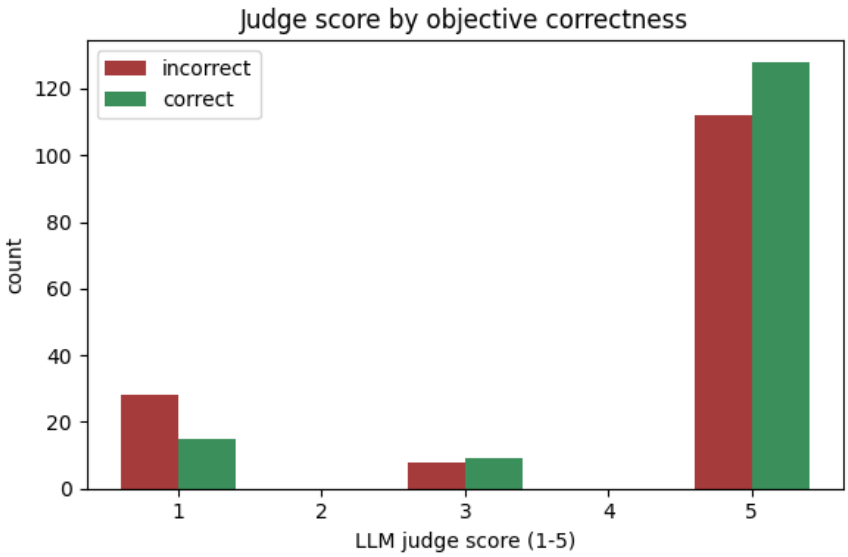

Figure 3: LLM-judge score by objective correctness: the judge over-rates wrong answers.

6.4 RQ3 — Latency and overhead

The **routing decision adds tens of microseconds** (B6 ≈26.5 μs/ request) — negligible vs. network latency. End-to-end latency tracks the chosen model and provider-side variance. Notably, B6's p95 (3475 ms) exceeds premium-static's (2494 ms), which we do not hide: B6 mixes models, giving a heavier-tailed latency distribution. We resolve whether this is real or single-pass noise in §6.9. Least-cost has the lowest latency (p95 1476 ms), consistent with always picking small models (Figure 4).

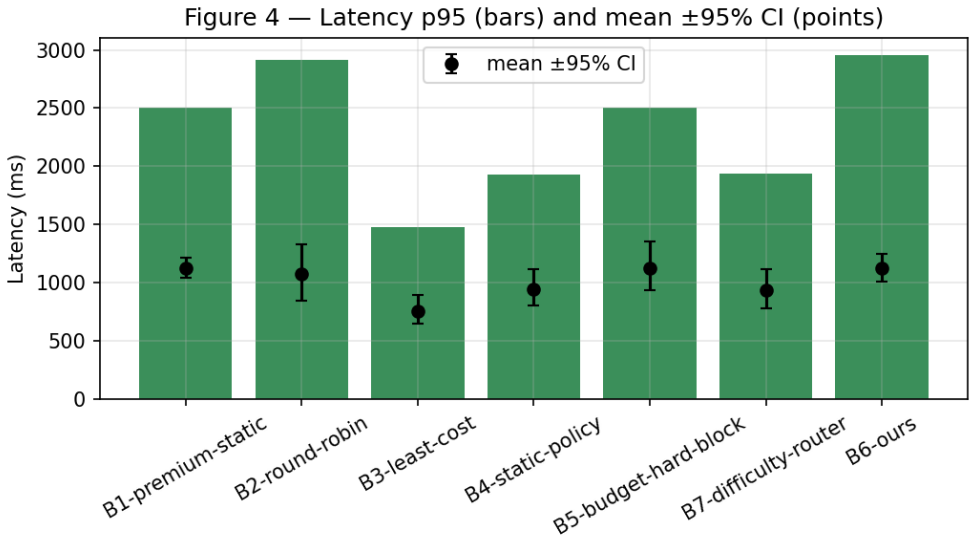

Figure 4: Latency percentiles by strategy.

Gateway data-path overhead (real Envoy, under load). Beyond control-plane decision time, we measured routing through a real **Envoy** data path: an Envoy proxy in front of the provider, driven at concurrency 8. Envoy adds **negligible overhead** — mean latency 1153 ms vs. 1116 ms direct (≈+3%, within run-to-run variance), throughput 6.05 vs. 5.81 req/s, **0 errors** — supporting the feasibility of a gateway-level control plane. Full in-cluster Envoy AI

Gateway integration remains future work.

6.5 RQ4 — Sovereignty

A sovereignty-blind baseline (B1) commits **40 violations** under eu-only, france-only and self-hosted-only, and 10 under no-external-sensitive. **B6-ours commits zero violations in every scenario.** Crucially, with a **real EU provider** (Mistral-Large-3, Azure AI Foundry DataZone EU), under the **EU-only** policy Ours **keeps 100% availability (40/40 served) with zero violations** by rerouting to Mistral EU (quality 0.866) — where the blind baseline would commit 40 violations. The *cost of sovereignty* appears in the stricter france-only/self-hosted-only scenarios: only the modeled FR self-hosted model qualifies, so 20/40 are served and the rest refused rather than violated — quantifying the availability price of strict residency (Figure 5).

Figure 5 — Sovereignty: cost & violations (B1 vs Ours)

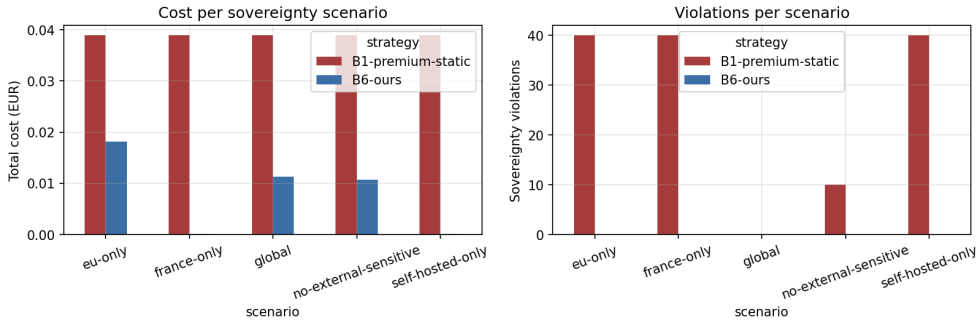

Figure 5: Sovereignty violations: blind baseline vs. Ours.

6.6 RQ5 — Budget-aware degradation

Under a budget tight enough to be exceeded by always-premium, **ours-graceful keeps 100% availability with 0% budget overrun** (quality 0.85), whereas **hard-block** falls to **60% availability** (4/10 requests blocked) and alert-only overshoots the budget by 150%. Graceful degradation dominates on availability *and* spend (Figure 6).

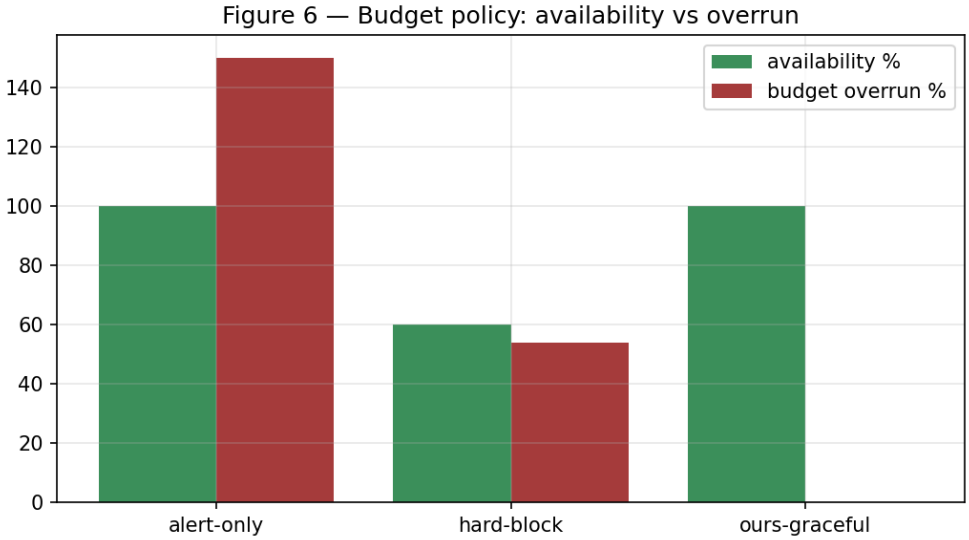

Figure 6: Budget policies: availability vs. overrun.

6.7 RQ6 — Managed vs. self-hosted break-even (*modeled*)

Using real per-token prices and modeled GPU+ops cost (2500 EUR/month, 5000 EUR migration), managed cost grows with volume while self-hosted is flat. The crossover is at ≈25.6**M tokens/day** (payback ≈4.3 months); above it the engine recommends *self-host*. This is a modeled prediction to be validated against real GPU economics (Figure 7).

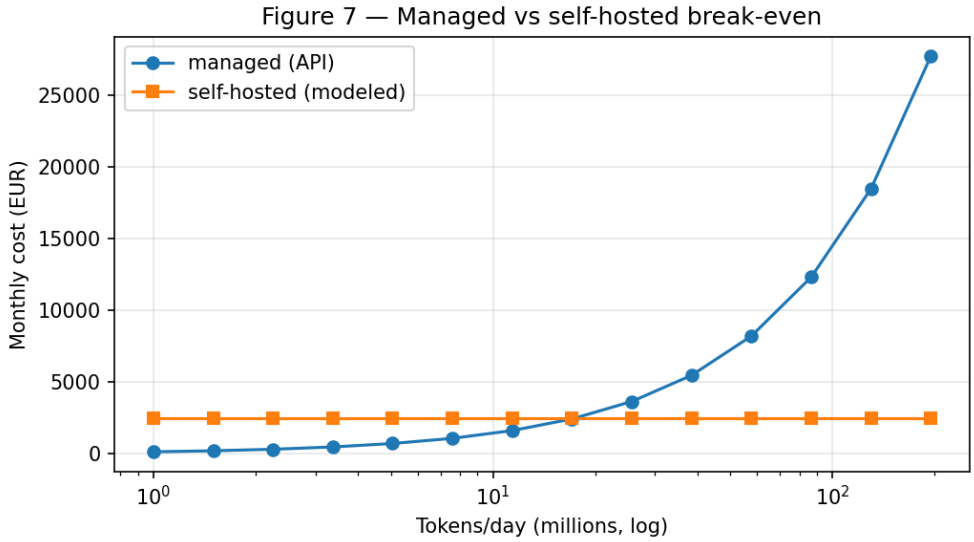

Figure 7: Managed vs. self-hosted cost vs. daily volume (modeled).

6.8 Ablation

Removing the **cost term** reverts spend to premium (0.0389 EUR) at unchanged quality — the cost term produces the saving. Removing the **quality term** drives spend to the floor (0.0012 EUR) but lowers quality to 0.863 — the quality term protects it. The latency term has negligible effect here. The full system attains the cost saving *and* the quality floor (Figure 8).

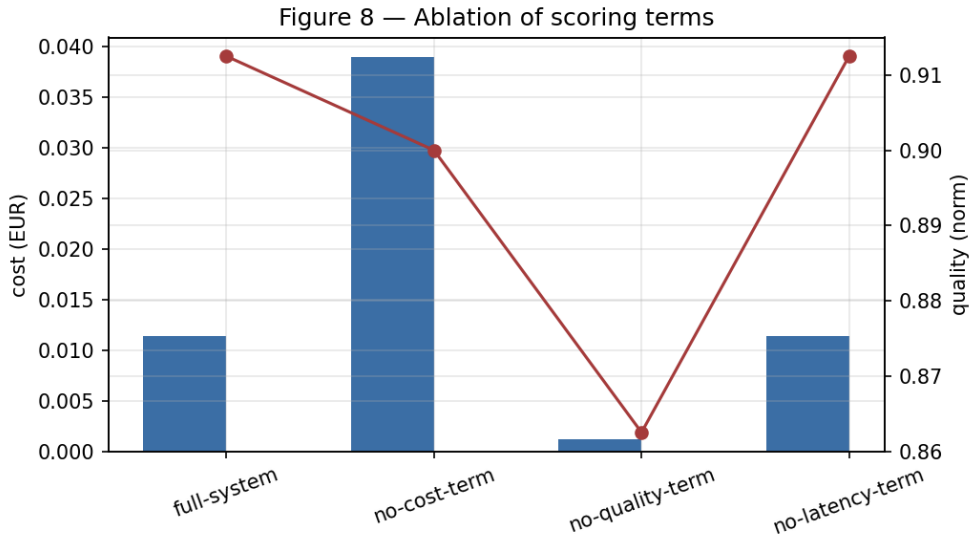

Figure 8: Ablation of scoring terms.

6.9 Statistical significance (N=30 repetitions)

To move beyond the single deterministic pass, we re-ran the full matrix **30 times** at temperature 0.7 with caches bypassed (independent samples; judge = gpt-4o-mini to bound cost), and tested each strategy against premium-static on the per-repetition distributions (Mann-Whitney U, Cliff's δ, Cohen's d, 95% CIs).

- **Cost.** Ours 0.0110 EUR/rep [0.0109, 0.0110] vs. premium 0.0385 [0.0382, 0.0389] — a **71.4% reduction**, $p \approx 3 \times 10^{-11}$, Cliff $\delta = -1.00$, Cohen $d = -43$ (non-overlapping CIs): highly significant with a maximal effect size.
- **Quality.** Ours mean normalized quality 0.9587 [0.9564, 0.9611] vs. premium 0.9362 — under this judge Ours is significantly higher ($p \approx 6 \times 10^{-11}$, $\delta = +0.97$, $d = +3.4$), while naïve least-cost is significantly lower (0.869, $d = -13$). The *sign* of the small quality gap depends on the judge (the single-pass gpt-4o judge showed parity), so we claim quality is *at least comparable*.
- **Latency.** Ours is significantly higher (1506 ms/rep [1459, 1553] vs. 998, $p \approx 5 \times 10^{-10}$, $d = +3.7$) — confirming the routing-mix latency cost is *real, not single-pass noise*. Least-cost is lowest (707 ms); the latency/cost trade-off is now quantified.

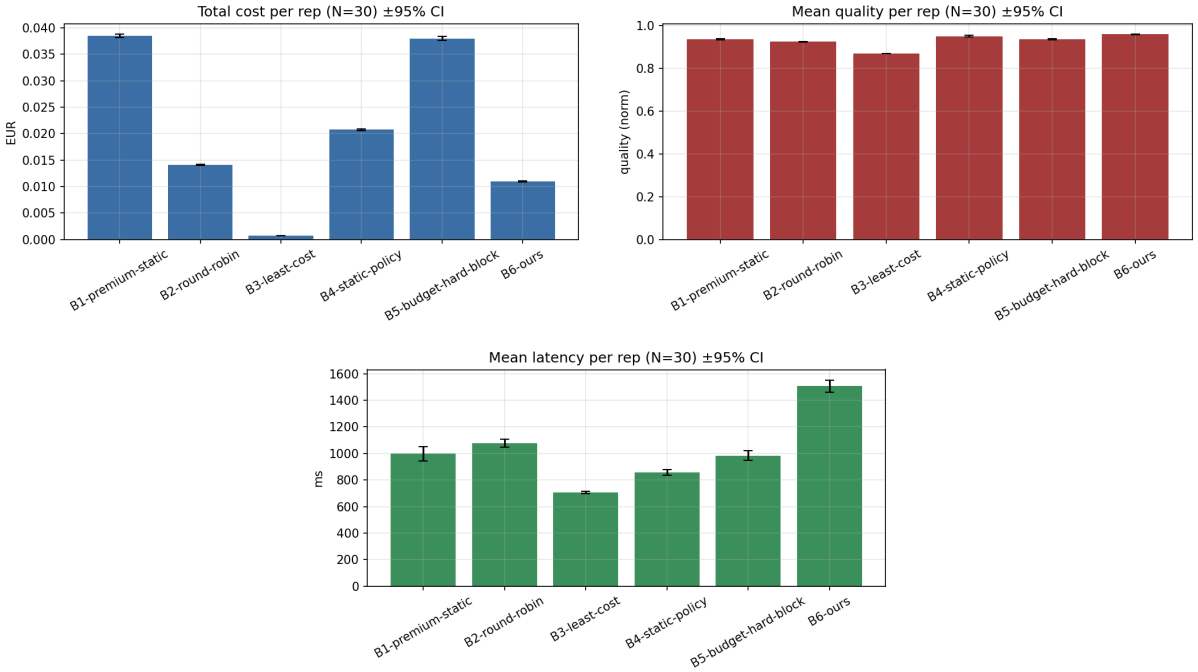

Figure 9: Per-repetition cost, quality and latency, mean ± 95% CI (N=30).

7 Discussion

Economic-aware routing is most valuable when a workload mixes easy and hard requests: the router sends cheap requests to cheap models and reserves the premium model for quality-critical ones, capturing most of the savings at little quality cost. Strict sovereignty has a real availability/quality price that the system makes explicit and auditable rather than hidden. For budget control, graceful degradation strictly dominates hard-blocking on availability and overrun. Self-hosting pays off only at high, sustained volume.

7.1 When *not* to use economic routing

Economic routing is not a universal win; our own results delimit where to avoid it:

- **High-stakes, correctness-critical tasks.** On objective reasoning (RQ2b) cheaper models genuinely fail, so cost routing trades accuracy for cost (47.2% vs. 65.4%). Where a wrong answer is expensive (legal, medical, shipping code, financial calculations), pin the premium model.
- **Low volume / low spend.** Below a few hundred thousand tokens/month the absolute saving is small and does not justify the added control-plane complexity; premium-static is simpler.
- **Strict tail-latency SLOs.** Ours shows a significant tail-latency increase (RQ3, $d{=}+3.7$) from mixing models; for ultra-low-latency paths, a single fast model is preferable.
- **No reliable quality signal.** Our priors miss task-specific difficulty and the LLM judge over-rates wrong answers (RQ2b); without a trustworthy per-task quality estimate, routing can silently degrade quality — prefer a fixed model or add a calibrated/learned difficulty predictor first.
- **No governance need.** If there is no sovereignty, budget, or attribution requirement (single tenant, single jurisdiction, flat budget), the governance layer adds little; a plain cost–quality router (or none) suffices.
- **Homogeneous high-quality workloads.** If essentially every request needs the premium model, there is nothing to route — the policy degenerates to premium-static.

In short, our approach targets **multi-tenant, governance-constrained, mixed-difficulty, sufficiently high-volume** settings; outside that envelope its benefits shrink or invert.

8 Threats to Validity

Internal: API/latency variability (mitigated by temperature 0, caching, bootstrap CIs); LLM-as-judge bias (absolute + pairwise, fixed judge). **External:** two providers and *modeled* self-hosting in this phase; four synthetic workloads; modeled GPU economics (RQ6); the Mistral EU model uses Azure AI Foundry DataZoneStandard (EU data zone) — strict France-only residency still relies on a modeled fallback; volatile cloud prices (isolated as configuration). **Construct:** acceptability/win-rate are quality proxies; declarative sovereignty simplifies legal reality — the system targets *audit preparation*, not legal compliance. **Conclusion:** beyond the deterministic pass we report a 30-repetition run (temperature 0.7) with 95% CIs, Mann-Whitney U significance and Cliff's δ/Cohen's d effect sizes; the cost reduction and the latency increase are highly significant with large effect sizes. The remaining caveat is *judge dependence* of the small quality gap. A two-judge agreement study (gpt-4o vs. Mistral-Large, $n{=}120$) finds *moderate* agreement — Cohen's quadratic-weighted $\kappa = 0.40$, Krippendorff $\alpha = 0.39$, Spearman $\rho = 0.61$ ($p \approx 10^{-13}$), within-1 agreement 94.2% — which is precisely why we treat the small quality gap as judge-dependent and plan a human evaluation (a blind 100-item human-eval package is prepared and awaits evaluators).

9 Conclusion and Future Work

Within our preliminary, two-provider testbed, a gateway-level, economic-aware, sovereignty-constrained, budget-aware control plane substantially reduced cost (-70.8%) while keeping

quality comparable, enforcing sovereignty with zero violations — including **maintaining 100% availability under an EU-only policy via a real EU provider** — and preserving availability under budget pressure. We deliberately frame these as *preliminary evidence and a reproducible framework*, not universal claims. To reach a defensible Q1 result we plan: (1) more providers; (2) real GPU self-hosting to validate the modeled break-even; (3) multi-judge + human quality evaluation; (4) human evaluation to settle the judge-dependent quality sign (the 100-item package is ready; the N=30 statistics and the judge-vs-ground-truth study are done); (5) data-path enforcement in Envoy with load testing and failover; (6) a human FinOps expert vs. automated control-plane study; and (7) integrating a learned difficulty router with our governance constraints. The paper claims only what the committed results support.

Artifact / Reproducibility. The operator and `experimentation/` (datasets, harness, scripts, results, figures, cached responses) constitute the reproducible artifact. Reproduce with `scripts/run_experime` then `python3 scripts/analyze_results.py`. Every test is journaled; no test is skipped.

References

[1] Pranjal Aggarwal, Aman Madaan, Ankit Anand, et al. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL `https://arxiv.org/abs/2310.12963`.

[2] BerriAI. LiteLLM: Call 100+ llm apis in the openai format. `https://github.com/BerriAI/litellm`, 2024. Open-source LLM gateway/proxy. Accessed 2026.

[3] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. 2023. URL `https://arxiv.org/abs/2305.05176`. arXiv:2305.05176.

[4] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://arxiv.org/abs/2404.14618`.

[5] Envoy Proxy Community. Envoy AI gateway: Unified access to generative AI services on envoy. `https://aigateway.envoyproxy.io/`, 2025. Open-source project (Bloomberg, Tetrate); v0.1.0 Feb 2025. Accessed 2026.

[6] European Union. Regulation (eu) 2016/679 (general data protection regulation). Official Journal of the European Union, 2016. `https://eur-lex.europa.eu/eli/reg/2016/679/oj`.

[7] European Union. Regulation (eu) 2024/1689 of the european parliament and of the council (artificial intelligence act). Official Journal of the European Union, 2024. Published 12 July 2024; in force 1 August 2024. `https://eur-lex.europa.eu/eli/reg/2024/1689/oj`.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165. URL `https://arxiv.org/abs/2309.06180`.

[9] Ranran Li et al. Taming the titans: A survey of efficient LLM inference serving. 2025. URL `https://arxiv.org/abs/2504.19720`.

[10] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. 2024. URL https://arxiv.org/abs/2406.18665. arXiv:2406.18665.

[11] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[12] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.05685.

[13] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2312.07104.

[14] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. URL https://arxiv.org/abs/2401.09670.

[15] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, et al. A survey on efficient inference for large language models. 2024. URL https://arxiv.org/abs/2404.14294.